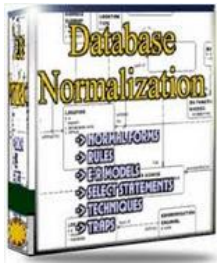
Technical University of Denmark

DTU Compute
Department of Applied Mathematics and Computer Science



Normalization

a database schema design technique to minimize data redundancy and avoid modification anomalies

Parts of Chapter 7 [8] of 7th [6th] Edition of the Textbook

©Anne Haxthausen and Flemming Schmidt

These slides have been prepared by Anne Haxthausen, partly reusing/modifying slides by Flemming Schmidt. Some examples come from Silberschatz, Korth, Sudarshan, 2010.

Contents

About Normalization

- *Normalization* is a simple, practical technique used to minimize data redundancy and avoid modification anomalies.
  - **What:** Normalization involves *decomposing a table into smaller tables* without losing information *and by defining foreign keys*.
  - **The objective** is to isolate data so that additions, updates and deletions can be made in just one place, and then the changes can be propagated through the rest of the database using the defined foreign keys. The **goal** is both to ensure *data consistency* and to *avoid extensive searches*.

Redundancy and Modification Anomalies

- Redundancy is the Evil of Databases
  - *Redundancy* means that the same data is stored in more than one place.
  - **The problem with redundancy is the risk of** *modification anomalies*: i.e. when data are modified (with SQL INSERT, UPDATE and DELETE commands), there is a risk that the data are not modified everywhere making the database inconsistent.
  - **The problem with an inconsistent database** is that it might return different answers to the same question asked in different ways.
  - If the database is redundancy free, then the DBMS can modify the data efficiently, without having to search the entire database.

Redundancy and Modification Anomalies

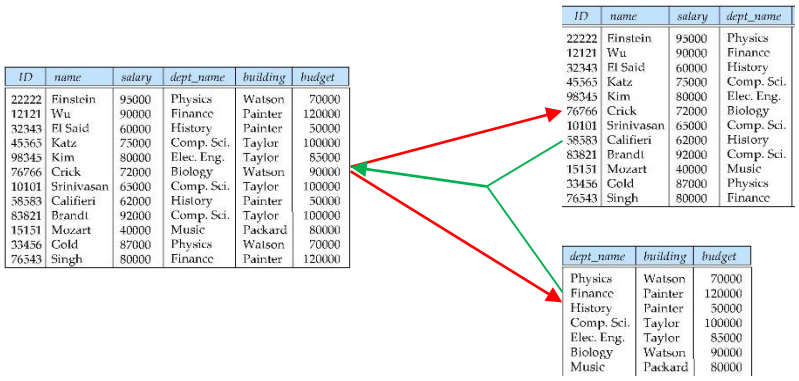
- Suppose we combine *instructor* and *department:*

ID	*name*	*salary*	*dept_name*	*building*	*budget*
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

- This design has two problems:
  - It has **redundant information**: building and budget are repeated for each instructor working in the same department. This can give rise to modification anomalies where the data becomes inconsistent.
  - It is not possible to store information about a department unless at least one instructor works in the department.

Features of Good Relational Design

- To avoid modification anomalies, additional restrictions can be imposed on tables/relation schemas.
- Tables/relation schemas with such restrictions are said to be in a given normal form like “Third Normal Form” or 3NF.
- Normalization is the process of bringing tables and their relation schemas to higher normal forms by avoiding redundancy. Normally it is done by projections of a table into two or more, smaller tables.
- Normalization is information preserving: it provides data lossless decompositions (usually projections), and is fully reversible, normally by joining the normalized tables back into the original table.



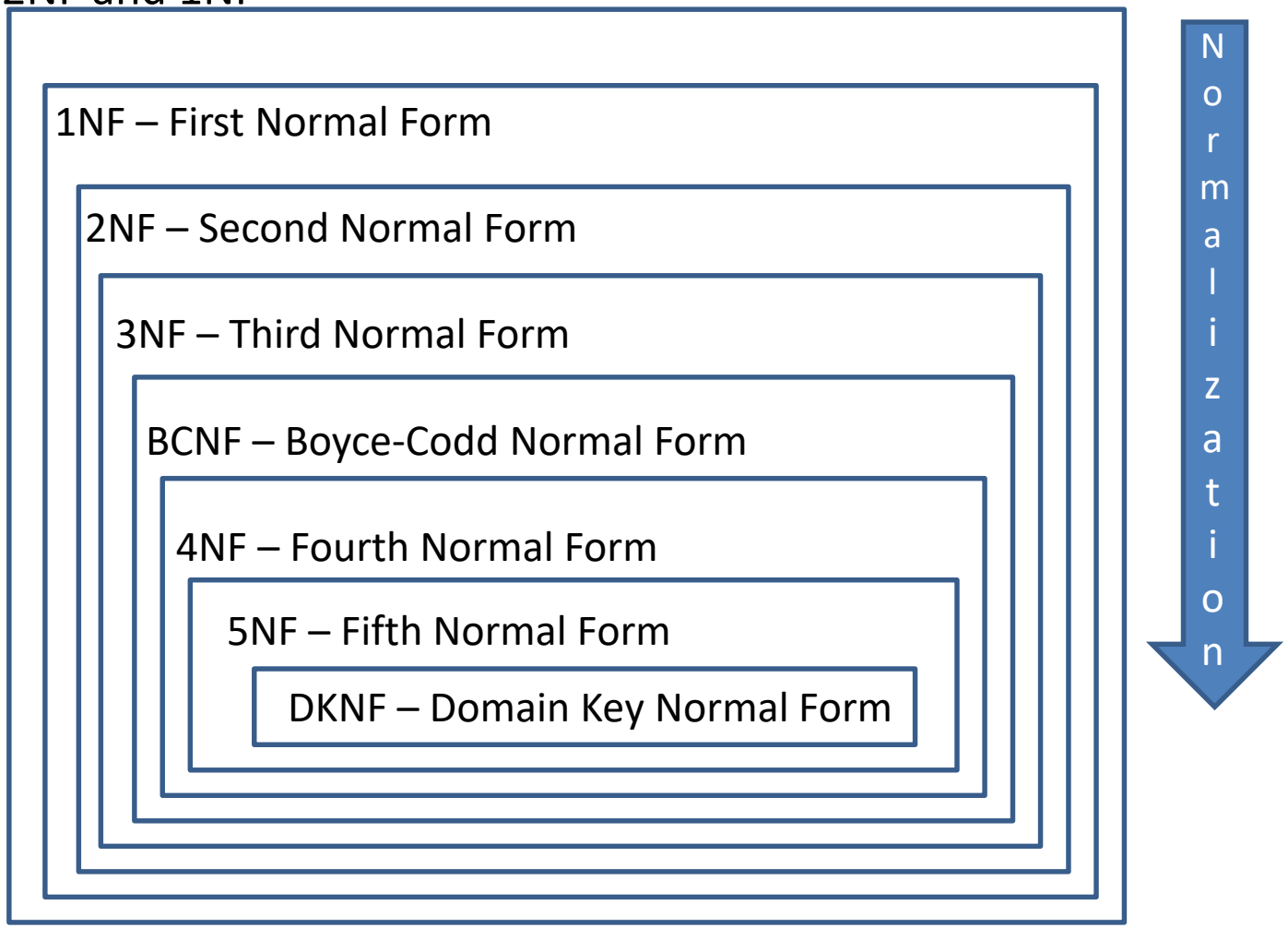
ID	*name*	*salary*	*dept_name*	*building*	*budget*
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

ID	*name*	*salary*	*dept_name*
22222	Einstein	95000	Physics
12121	Wu	90000	Finance
32343	El Said	60000	History
45565	Katz	75000	Comp. Sci.
98345	Kim	80000	Elec. Eng.
76766	Crick	72000	Biology
10101	Srinivasan	65000	Comp. Sci.
58583	Califieri	62000	History
83821	Brandt	92000	Comp. Sci.
15151	Mozart	40000	Music
33456	Gold	87000	Physics
76543	Singh	80000	Finance

dept_name	*building*	*budget*
Physics	Watson	70000
Finance	Painter	120000
History	Painter	50000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Biology	Watson	90000
Music	Packard	80000

Normal Form Hierarchy

- If a table/relation schema is in e.g. 3NF, then by definition it is also in 2NF and 1NF



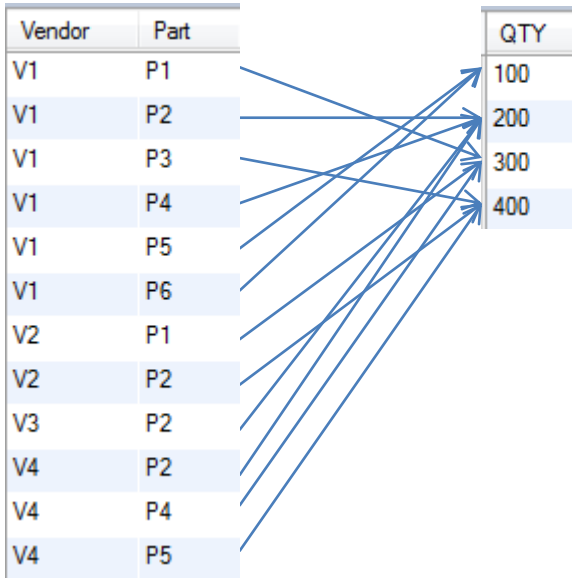
Theoretical Foundations: Functional Dependencies

- Mathematics needed to define 1NF, 2NF, 3NF and BCNF
  - *Functional dependencies:* attribute associations within a relation (schema).
    - Trivial and nontrivial functional dependencies
    - Keys defined in terms of functional dependencies
    - Armstrong's axioms and derived theorems: to find *derived functional dependences*
- Mathematics needed to define 4NF (later)
  - *Multivalued dependencies.*

Functional Dependencies

- Describe dependencies between attribute sets A and B of a relation schema R:
  - Basically *a many-to-one relation* of associations from one set A of attributes to another set B of attributes within a given relation.
- Example: Consider Shipments(Vendor, Part, Qty):

Vendor	Part	Qty
V1	P1	300
V1	P2	200
V1	P3	400
V1	P4	200
V1	P5	100
V1	P6	100
V2	P1	300
V2	P2	400
V3	P2	200
V4	P2	200
V4	P4	300
V4	P5	400



{Vendor, Part} → {Qty} is *a functional dependency* **holding for** Shipments.

The attribute set {Vendor, Part} *functionally determines* the attribute set {Qty}.

The attribute set {Qty} is *functionally dependent* of the attribute set {Vendor, Part}.

Formal Definition of Functional Dependency

Let *X* and *Y* be subsets of the set of attributes of a relation schema *R* .

- A functional dependency $X \rightarrow Y$ **holds** for R if and only if
  - in every legal instance of *R*,
    each *X* value has associated precisely one *Y* value (i.e. whenever two rows have the same *X* value, they also have the same *Y* values).
- When $X \rightarrow Y$ **holds** for R, we say
  - *Y is functionally dependent of X* and *X functionally determines Y*
  - *X* is said to be the *determinant* and *Y* the *dependent.*
  - If $X = \{A_1, \ldots, A_n\}$, $Y = \{B_1, \ldots, B_m\}$ we sometimes write $A_1, \ldots, A_n \rightarrow B_1, \ldots, B_m$ or $A_1 \ldots A_n \rightarrow B_1 \ldots B_m$
- Trivial Dependencies
  - $X \rightarrow Y$ is *trivial*, if $Y \subseteq X$.
  - Example 1: {*Vendor, Part*} → {*Vendor*}
  - Example 2: {*Part*} → {*Part*}
- Nontrivial Dependencies
  - Those FDs which are not trivial.
  - Nontrivial FDs lead to definitions of integrity constraints like keys.

Functional Dependencies

- Many functional dependencies can be proposed.
- Example: Shipment1(Vendor, City, Part, Qty) (where a vendor only can be in one city)

Vendor	City	Part	Qty
V1	London	P1	100
V1	London	P2	100
V2	Paris	P1	200
V2	Paris	P2	200
V3	Paris	P2	300
V4	London	P2	400
V4	London	P4	400
V4	London	P5	400

No	Determinant	FD	Dependent	Validity	Remark
1	{*Vendor, Part*}	→	{*Qty*}	Legal	
2	{*Vendor*}	→	{*City*}	Legal	
3	{*Qty*}	→	{*Vendor*}	Illegal	
4	{Part}	→	{*Qty*}	Illegal	
5	{*Vendor, Part*}	→	{*City*}	Legal	Derived(2)
6	{*Vendor*}	→	{*Vendor*}	Legal	trivial
256		→			

- To decide whether a FD is *valid* (legal for all relation instances), one has to consider the real world.
- Of a set of 4 attributes, it is possible to make $2^4 = 16$ subsets.
- Combining 16 determinants with 16 dependents gives 256 potential FDs.
- Of the 256 potential FDs, some are valid and some are not.
- **(Canonical) Cover Set:** A (irreducible/minimal) set of valid functional dependencies from which all valid functional dependencies can be determined. In the example: { {Vendor, Part} → {Qty}, {Vendor} → {City}} is a canonical cover set.
- **Closure Set F+ of a set F of functional dependencies**: The set of all valid functional dependencies that can be logically derived from the F.

Super Key and Functional Dependencies

Let R be a relation schema and let K be a subset of the set A_R of attributes of R.

- Super key, original definition:
  - K is a *superkey* of R, if, in any legal instance of R, for all pairs t_1 and t_2 of tuples in the instance of R if $t_1 \neq t_2$, then $t_1[K] \neq t_2[K]$. Hence, a specific K value will uniquely define a specific tuple in R.
- Super key defined by functional dependencies:
  - K is a *superkey* of R, if $K \rightarrow X$ holds for *every* attribute X of R.
  - Super Key examples for Shipment1(Vendor, City, Part, Qty)
    - {Vendor, City, Part}
      Knowing the values of Vendor, City and Part, the value of Qty is defined.
    - However, of interest is a Super Key with a minimum set of attributes:
      {*Vendor*, *Part*}
      Knowing the values of Vendor and Part, the values of City and Qty is defined.

Candidate Key and Functional Dependencies

- Candidate key is a minimal super key:
  - *K* is a *candidate key* for *R* if and only if,

    $K \rightarrow A_R$ *and* for no $\alpha \subset K$, $\alpha \rightarrow A_R$
  - Candidate key example for Shipment1(Vendor, City, Part, Qty): {*Vendor*, *Part*}
  - In some cases *R* can have several candidate keys!

- Primary key
  - A candidate key is selected by the DBA to be the *primary key* for a relation.

Armstrong's Rules

Let R be a relation schema and let F be a set of functional dependencies on R.

- Armstrong's Rules
  - are some Axioms and Derived Theorems for deriving functional dependences from F
  - are
    - **sound**: only valid FDs are derived from valid FDs.
    - **complete**: all valid FDs (the closure set of *F*) can be derived from a cover set *F*.
  - can e.g. be used for finding candidate keys

Armstrong's Axioms and Derived Theorems

Let X, Y, Z and V be sets of attributes.

- Armstrong's axioms
  1. Reflexivity: If Y is a subset of X, then $X \rightarrow Y$
  2. Augmentation: If $X \rightarrow Y$, then $XZ \rightarrow YZ$
     Note: XZ is used as a shorthand for $X \cup Z$
     Note: $XY = YX$ and $XX = X$; Sets have no order and no repetitions
  3. Transitivity: If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$
- Derived theorems
  4. Self-determination: $X \rightarrow X$
  5. Decomposition: If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$
  6. Union: If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$
  7. Composition: If $X \rightarrow Y$ and $Z \rightarrow V$, then $XZ \rightarrow YV$
  8. General Unification: If $X \rightarrow Y$ and $Z \rightarrow V$, then $X \cup (Z - Y) \rightarrow YV$
     (where '∪' is Set Union and '–' is Set Difference)

Using Armstrong's Rules

- To propose candidate keys
  - Given R(A, B, C, D, E) with functional dependencies: A → BC, CD → E, B → D, E → A

1.	A → A	Rule 4 Self-determination
2.	A → B	Rule 5 Decomposition of given A → BC
3.	A → C	Rule 5 Decomposition of given A → BC
4.	A → D	Rule 3 Transitivity of A → B and given B → D
5.	A → CD	Rule 6 Union A → CD
6.	A → E	Rule 3 Transitivity of A → CD and given CD → E
7.	A → ABCDE	Rule 6 Union
8.	E → ABCDE	Rule 3 Transitivity of given E → A and A → ABCDE
9.	BC → ABCDE	can also be proved
10.	CD → ABCDE	can also be proved
11.	Candidate Keys:	Both {A}, {E}, {B,C}, and {D,C}. Select one for the PK!
12.	Relation Schema:	one of following: R(<u>A</u>, B, C, D, E), R(<u>E</u>, A, B, C, D), R(<u>B</u>, <u>C</u>, A, D, E), or R(<u>C</u>, <u>D</u>, A, B, E)

Normal Forms 1NF-3NF: Original Definitions

- **2NF** and **3NF** are defined using the notion of *functional dependencies*.

  The starting point for the definitions is:
  - a relational schema R
  - a cover set FD of functional dependencies for R
- **The original definitions** (here) assume the tables have one candidate key which has been chosen as *primary key*, in the following called the *key*.
- **The generalized definitions** of **2NF** and **3NF** (in the book) generalizes the original definitions by taking *several candidate keys* into account.
- When there is only one candidate key, the original and the generalized definitions are equivalent.

Normal Forms Defined Informally

- 1st normal form
  - All attributes depend on **the key.**
- 2nd normal form
  - All attributes depend on **the whole key**
- 3rd normal form
  - All attributes depend on **nothing but the key**

1NF – First Normal Form

- For a relation to be in First Normal Form 1NF
  1. Each attribute value must be a single value (is atomic, not multivalued or composite).

OrderNo	ItemNo	Qty
1000	100	18
1000	102	13
1000	105	24
1001	102	24
1002	100	14
1002	105	21

Normalization to 1NF

- *OrdersTable* below is not in 1NF

  as the values of the attribute ItemNos are not atomic.

OrdersTable(OrderNo, ItemNos)

OrderNo	ItemNoS
1000	100,102,105
1001	102
1002	100,105

- Normalization to 1NF

Orders1NF(OrderNo, ItemNo)

OrderNo	ItemNo
1000	100
1000	102
1000	105
1001	102
1002	100
1002	105

2NF – Second Normal Form

- For a relation schema R to be in Second Normal Form 2NF
  1. It must be in 1NF.
  2. Each non primary key attribute A must not depend on a strict subset K_{part} of the primary key attribute set K, i.e. K_{part} -> A and $K_{part} \subset K$ must not be the case. A must depend on the entire primary key K.
- Some special cases where a 1NF relation schema is in 2NF:
  - The primary key consists only of one attribute.
  - The primary key consists of all attributes in the relation.
- Normalisation 1NF to 2NF:
  - Move the set of all attributes A which depend on a $K_{part} \subset K$ to a new relation R2 together with a copy of K_{part}, which becomes the primary key as well as a foreign key. If there is only one such attribute A we have:

    R(<u>K</u>, …, A) ⇨ R1(<u>K</u>, …) foreign key K_{part} references R2, R2(<u>K_{part}</u>, A)
  - Repeat the step above, if R1 or R2 are not yet in 2 NF.
  - Decomposition is *data lossless*:

    (**select** K, … **from** R) **natural join** (**select** K_{part}, A **from** R) = **select** * **from** R

Normalization to 2NF - Example

- *Orders1NF* is in 1NF, but not 2NF:

OrderNo	ItemNo	ItemName
1000	100	Car Nitromethan
1000	102	Airplane Spitfire
1000	105	Battleship Bismarck
1001	102	Airplane Spitfire
1002	100	Car Nitromethan
1002	105	Battleship Bismarck

- Orders1NF(OrderNo, ItemNo, ItemName) with FD: ItemNo -> ItemName
- ItemName depends on ItemNo only, and not on the full primary key. Violation of rule 2 for 2NF.

- Normalization to (*Orders2NF* and *Items* in) 2NF
  - Orders2NF(OrderNo, ItemNo) foreign Key(ItemNo) references Items(ItemNo)
  - Items(ItemNo, ItemName)
  - Note that ItemNo is included in the key of Orders2NF, as OrderNo -/-> ItemNo

OrderNo	ItemNo
1000	100
1000	102
1000	105
1001	102
1002	100
1002	105

ItemNo	Itemname
100	Car Nitromethan
102	Airplane Spitfire
105	Battleship Bismarck

- Associated normalization of tables is projections:
  Orders2NF ≡ $\prod_{OrderNo,\ ItemNo}$ (Orders1NF)
  Items ≡ $\prod_{ItemNo,\ ItemName}$ (Orders1NF)
- A Natural Join brings back the table:
  Orders1NF ≡ Orders2NF ⋈ Items
- Normalization is information preserving .

3NF – Third Normal Form

- For a relation schema R(K, …, A) to be in Third Normal Form 3NF
  1. It must be in 2NF.
  2. Each non primary key attribute A must depend ***directly*** on the entire primary key K. It must not depend ***transitively*** via other attributes B, like K -> B -> A, where B -/-> K and B -> A is non-trivial (i.e. $A \not\subseteq B$).
- Normalization 2NF to 3NF
  - Move attributes A which are transitively dependent on K via B, i.e. K -> B -> A for some attribute set B, to a new relation R2 together with a copy of the dependent attributes in B, which become the primary key and constitute a foreign key:

    R(K, …, B, A) ⇨ R1(K, …, B) foreign key B references R2, R2(B, A)
  - Repeat the step above, if R1 or R2 are not yet in 3 NF.
  - Decomposition is *data lossless*:

    (**select** K, …, B **from** R) **natural join** (**select** B, A **from** R) = **select** * **from** R

Normalization to 3NF - Example

- *Customer2NF* in 2NF

CustomerNo	PostNo	CityName
10500	2500	Valby
10501	2500	Valby
10502	2600	Glostrup
10503	2500	Valby
10504	2605	Brøndby
10505	2600	Glostrup

- Customers2NF(CustomerNo, PostNo, CityName)
- CustomerNo -> PostNo, PostNo -> CityName
- CityName depends on the full Primary Key, but transitively via PostNo. Violation of rule 2 for 3NF.

- Normalization to 3NF

CustomerNo	PostNo
10500	2500
10501	2500
10502	2600
10503	2500
10504	2605
10505	2600

PostNo	CityName
2500	Valby
2600	Glostrup
2605	Brøndby

- Customers3NF(CustomerNo, PostNo), foreign key(PostNo) references Post(PostNo)
- Post(PostNo, CityName)
- Associated normalization of tables is projections:
  $\text{Customer3NF} \equiv \prod_{\textit{CustomerNo, PostNo}} (\text{Customer2NF})$
  $\text{Post} \equiv \prod_{\textit{PostNo, CityName}} (\text{Customer2NF})$
- A Natural Join brings back the table:
  $\text{Customer2NF} \equiv \text{Customer3NF} \bowtie \text{Post}$
- Normalization is information preserving.

Normal Forms 2NF-3NF, BCNF: General Definitions

- Are defined using the notion of *functional dependencies*.
- **The original definitions** (on previous slides) of 2NF-3NF assume the tables have one candidate key which has been chosen as *primary key*.
- **The generalized definitions** of 2NF-3NF and BCNF (in the book) take *several candidate keys* into account.
- When there is only one candidate key, the original and the generalized definitions of 2NF-3NF are equivalent.
- When there is only one candidate key: 3NF and BCNF are the same.
- 3NF and BCNF are the same (according to Date) unless
  - there are several composite candidate keys CK1 and CK2
  - which are overlapping ($CK1 \cap CK2 \neq \{\}$)

  This exception is very rare.

2NF-3NF General Definitions

- For a relation schema R to be in Second Normal Form 2NF
  1. It must be in 1NF.
  2. Each non candidate key attribute A must not depend on a strict subset K_{part} of any candidate key attribute set K, i.e. K_{part} -> A and $K_{part} \subset K$ must not be the case. A must depend on the entire K.
- For a relation schema R(K, …, A) to be in Third Normal Form 3NF
  1. It must be in 2NF.
  2. No non candidate key attribute A depends ***transitively*** on any candidate key K via other attributes B, like K -> B -> A, where B -/-> K and B -> A is non-trivial (i.e. $A \not\subseteq B$).

In the book there is another general definition of 3NF. The two definitions are equivalent.

BCNF - Boyce-Codd Normal Form

Boyce-Codd Normal Form BCNF

- A relation schema is in Boyce-Codd Normal Form BCNF, if and only if, every nontrivial, left-irreducible FD $\alpha \rightarrow \beta$ has a candidate key as its determinant α.
- *Left-irreducible* means that there is no proper subset s of α such that $s \rightarrow \beta$.

Normalization of $R(A_1, \ldots, A_n, B_1, \ldots, B_n, C)$ to BCNF:

- Assume $B_1, \ldots, B_n \rightarrow C$ is nontrivial, left-irreducible, but $B_1, \ldots, B_n$ is not a candidate key.
- $R(A_1, \ldots, A_n, B_1, \ldots, B_n, C)$ 

  $R_1(A_1, \ldots, A_n, B_1, \ldots, B_n)$ foreign key $B_1, \ldots, B_n$ references R_2 and $R_2(\underline{B_1}, \ldots, \underline{B_n}, C)$
- Repeat the step above, if R1 or R2 are not yet in BCNF.

Normalization to BCNF, example:

- *$R(\underline{A}, B, C)$ with $FD = \{A \rightarrow B, B \rightarrow C\}$*
  is not in BCNF ($B \rightarrow C$, but B is not candidate key)
- Decomposition of R: *$R_1(\underline{A}, B)$ and $R_2(\underline{B}, C)$.*

BCNF versus 3NF - Example

Given functional dependencies

Pizza, ToppingType → Topping

Topping → ToppingType

on R(Pizza, ToppingType, Topping)

Then R has two candidate keys:

- Pizza, ToppingType
- Pizza, Topping

The first is chosen as primary key.

Then

R(Pizza, Topping, ToppingType)

is in 3NF, but not in BCNF as Topping → ToppingType and Topping is not a candidate key.

Pizza	Topping	ToppingType
1	mozarella	cheese
1	pepperoni	meat
1	olives	vegetable
2	mozarella	cheese
2	sausage	meat
2	pebbers	vegetable

Higher Normal Forms: 4NF, 5NF, …

- Are defined using the notion of *multivalued dependencies*.
- Fourth Normal Form

  A relation schema with *multiple unrelated multivalued dependencies* must be broken into relation schemas that separate the unrelated attributes.
- Fifth Normal Form neither covered in the book nor here.

Multivalued Dependencies

Let R be a relation schema and α and β be disjoint subsets of the attributes of R and let Υ be the remaining attributes of R.

- A *multivalued dependency* $\alpha \twoheadrightarrow \beta$ **holds** for R if and only if, in every legal instance of R, the set of β values matching a given $\alpha\ \Upsilon$ value pair depends only on the α value and is independent of the Υ value.
- When $\alpha \twoheadrightarrow \beta$ we say α *multivalue determines* β.
- When $\alpha \twoheadrightarrow \beta$ **holds** for R, then $\alpha \twoheadrightarrow \Upsilon$ also **holds** for R.
- When $\alpha \rightarrow \beta$ **holds** for R, then $\alpha \twoheadrightarrow \beta$ also **holds** for R.

	α	β	Υ
t_1	$a_1 \ldots a_i$	$a_{i+1} \ldots a_j$	$a_{j+1} \ldots a_n$
t_2	$a_1 \ldots a_i$	$b_{i+1} \ldots b_j$	$b_{j+1} \ldots b_n$
t_3	$a_1 \ldots a_i$	$a_{i+1} \ldots a_j$	$b_{j+1} \ldots b_n$
t_4	$a_1 \ldots a_i$	$b_{i+1} \ldots b_j$	$a_{j+1} \ldots a_n$

Multivalued Dependencies - Example

- Example
  - CTX(Course, Teacher, Text)
    Course →→ Teacher 'Physics' →→ 'Prof. Green' and 'Prof. Brown'
    Course →→ Text 'Physics' →→ 'Principles of Optics' and 'Basic Mechanics'

Course	Teacher	Text
Math	Prof. Green	Basic Mechanics
Math	Prof. Green	Trigonometry
Math	Prof. Green	Vector Analysis
Physics	Prof. Brown	Basic Mechanics
Physics	Prof. Brown	Principles of Optics
Physics	Prof. Green	Basic Mechanics
Physics	Prof. Green	Principles of Optics

4NF - Fourth Normal Form

- Fourth Normal Form 4NF
  - A relation schema *R* is in Fourth Normal Form 4NF, if and only if, whenever

    a non-trivially $\alpha \rightarrow\!\!\!\!\rightarrow \beta$ holds, then α is a key of *R* (all attributes of *R* are also functionally dependent on α).

    $\alpha \rightarrow\!\!\!\!\rightarrow \beta$ is trivial means $\beta \subseteq \alpha$ or $\alpha \cup \beta$ = set of all attributes in *R*.

- Normalization from BCNF to 4NF
  - Assume $\alpha \rightarrow\!\!\!\!\rightarrow \beta$ non-trivially holds and α is NOT a key of *R.*
  - This usually arises from many-to-many relationship sets or multivalued entity sets.
  - *R*(α, β, Υ) ⇨ *R1*(α, β), *R2*(α, Υ)
- Decomposition is *data lossless*:

  (**select** α, β **from** R) **natural join** (**select** α, Υ **from** R) = **select** * **from** R

Multivalued Dependencies and 4NF

- Example
  - CTX(Course, Teacher, Text)
    Course →→ Teacher 'Physics' →→ 'Prof. Green' and 'Prof. Brown'
    Course →→ Text 'Physics' →→ 'Principles of Optics' and 'Basic Mechanics'
  - CTX is in 3NF, but not in 4NF as Course is not a key!

Course	Teacher	Text
Math	Prof. Green	Basic Mechanics
Math	Prof. Green	Trigonometry
Math	Prof. Green	Vector Analysis
Physics	Prof. Brown	Basic Mechanics
Physics	Prof. Brown	Principles of Optics
Physics	Prof. Green	Basic Mechanics
Physics	Prof. Green	Principles of Optics

Course	Teacher
Physics	Prof. Green
Physics	Prof. Brown
Math	Prof. Green

Course	Text
Physics	Principles of Optics
Physics	Basic Mechanics
Math	Vector Analysis
Math	Trigonometry
Math	Basic Mechanics

  - Decompose to reach 4NF:
    CT(Course, Teacher), CX(Course, Text)

    These are in 4NF, as Course →→ Teacher resp. Course →→ Text now are trivial
  - A Natural Join of CT and CX over Course will restore CTX

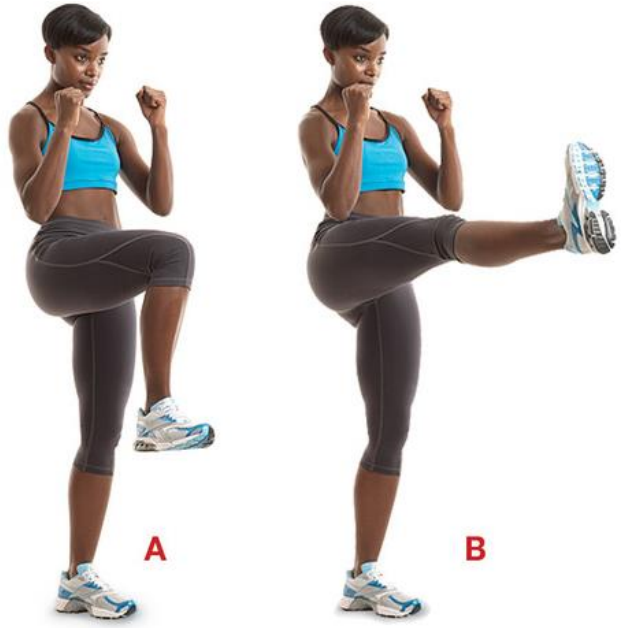
Summary

- Redundancy is the evil of all databases leading to modification anomalies (i.e. problems with SQL INSERT, UPDATE and DELETE).
- A good relational design avoids redundancy by decomposition of tables into multiple tables without redundancy.
- Decomposition of tables are done by data lossless projections and a natural join of the decomposed tables will bring back the original table.
- The decomposition process is called normalization, and the tables and their relation schemas gradually continues to improve in quality in higher and higher normal forms.

- Supplementary Literature:

  An Introduction to Database Systems, C.J. Date, Addison-Wesley, Eight Edition, 2004, Chapters 11, 12 & 13.

Demo Exercises



Demo Exercises clarify ideas and concepts from the Lecture to provide you with good Database Skills.

Discuss and do the Demo Exercises.

Functional Dependencies

7.1.1 Number of Functional Dependencies

How many functional dependencies can be proposed for the relation schema R(A, B, C)?

7.1.2 Candidate Keys with Armstrong's Rules

Use Armstrong's rules to make a list of Candidate Keys for the following relation schema R(A, B, C, D) with the following functional dependencies:
B → AD, D → B, A → C. Please specify which of Armstrong's rules are used in each step.
Finally select a Primary Key.

First Normal Form

7.1.3 Violation of First Normal Form 1NF

What is the problem, and how can the table below be normalized?

employee_no	Phone_numbers
67810101	28801633, 28801724
67810210	28801325, 28805647, 28801518
67810324	28801713

Third Normal Form

7.1.4 Violation of Third Normal Form 3NF

What is the problem, and how can the table below be normalized?

Driver_license	License_plate	Full_name	Car_manufacturer
31261435	JW46476	Jens Hansen	Honda
31237248	AX24583	Finn Jensen	Honda
31229753	AZ26184	Anna Flink	Mazda
31237248	KC27534	Finn Jensen	Honda
31231212	JW46538	Karin Holst	Mazda

BCNF

7.1.5 Violation of BCNF

Consider the table

Pizza	Topping	ToppingType
1	mozarella	cheese
1	pepperoni	meat
1	olives	vegetable
2	mozarella	cheese
2	sausage	meat
2	pebbers	vegetable

with relation schema
R(Pizza, Topping, ToppingType)

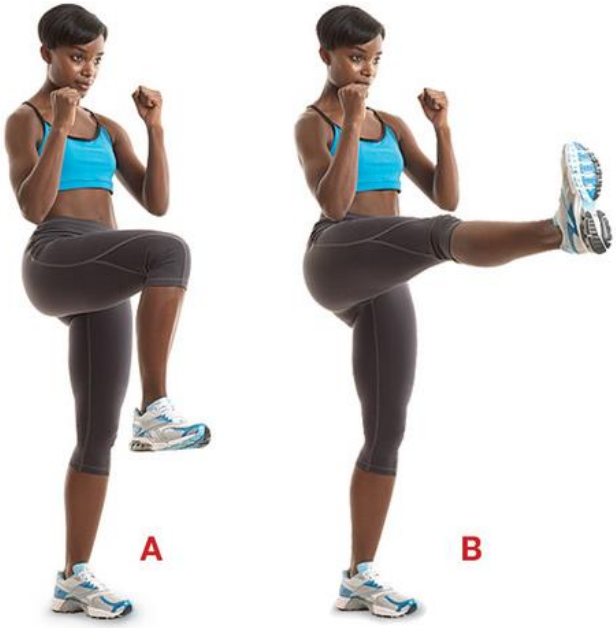
and functional dependencies
Pizza, ToppingType $\rightarrow$ Topping

Topping $\rightarrow$ ToppingType

Normalize the relation schema and table to BCNF.

7.1.5 Solution:

Solutions to Demo Exercises



Functional Dependencies

7.1.1 Number of Functional Dependencies

How many functional dependencies can be proposed for the relation schema R(A, B, C)?

7.1.1 Of a set of 3 attributes, $2^3 = 8$ subsets can be made (i.e. {}, {A}, {B}, {C}, {A,B}, {A,C}, {B,C}, and {A, B, C}. Each subset can be a Determinant or Dependent, thus 8 x 8 = 64 functional dependencies can be proposed, like {A, B} → {C}, {A} → {B}.

7.1.2 Candidate Keys with Armstrong's Rules

Use Armstrong's rules to make a list of Candidate Keys for the following relation schema R(A, B, C, D) with the following functional dependencies:
B → AD, D → B, A → C. Please specify which of Armstrong's rules are used in each step. Finally select a Primary Key.

7.1.2 By using Armstrong's rules:
1. B → B by rule 4 Self-determination
2. B → A by rule 5 Decomposition of B → AD
3. B → D by rule 5 Decomposition of B → AD
4. B → C given B → A ^ A → C by rule 3 Transitivity
5. B → ABCD by rule 6 Union
6. D → ABCD given D → B and B → ABCD by rule 3

Candidate Keys are B and D!
I select B to be the Primary Key, but I could have selected D instead.

First Normal Form

7.1.3 Violation of First Normal Form 1NF

What is the problem, and how can the table below be normalized?

employee_no	Phone_numbers
67810101	28801633, 28801724
67810210	28801325, 28805647, 28801518
67810324	28801713

7.1.3 It looks like the table has *employee_no* as primary key.

The normalization could be the one shown below, with a primary key

- (*employee_no*, *Phone_no*), if one phone number can be shared between several *employees.*
- *Phone_no,* if a phone number can belong only to one employee:

employee_no	Phone_no
67810101	28801633
67810101	28801724
67810210	28801325
67810210	28805647
67810210	28801518
67810324	28801713

As can be seen, there is no limit to the number of phone numbers that an employee can have, and a search of an employee's phone numbers is simple.

Third Normal Form

7.1.4 Violation of Third Normal Form 3NF

What is the problem, and how can the table below be normalized?

Driver_license	License_plate	Full_name	Car_manufacturer
31261435	JW46476	Jens Hansen	Honda
31237248	AX24583	Finn Jensen	Honda
31229753	AZ26184	Anna Flink	Mazda
31237248	KC27534	Finn Jensen	Honda
31231212	JW46538	Karin Holst	Mazda

7.1.4 Inspecting the tables and using domain knowledge we identify the following cover set of functional dependencies:

Driver_license -> Full_name

License_plate -> Driver_License

License_plate -> Car_manufacturing

Using Armstrong's transitivity rule we also have

License_plate -> Full_name

As {License_plate} functionally determines all attributes and is minimal, License_plate can be chosen as primary key. The Table is in 2NF as all attributes depend fully on the key, but not in 3NF as *Full_name* transitively depends on the key via *Driver_license*. The problem by this is that the information that 'Finn Jensen' has driver license '31237248' is recorded twice (i.e. redundant), and an update of *Driver_license* for 'Finn Jensen' might lead to ambiguity. Normalization gives the two 3NF tables below. The first has *driver_license* and the second has *license_ plate* as primary key. A natural join of the two (join over attribute *driver_license*), will bring back the original table.

driver_license	full_name
31229753	Anna Flink
31231212	Karin Holst
31237248	Finn Jensen
31261435	Jens Hansen

license_plate	driver_license	car_manufacturer
AZ26184	31229753	Mazda
JW46538	31231212	Mazda
AX24583	31237248	Honda
KC27534	31237248	Honda
JW46476	31261435	Honda

BCNF

7.1.5 Violation of BCNF

Consider the table

Pizza	Topping	ToppingType
1	mozarella	cheese
1	pepperoni	meat
1	olives	vegetable
2	mozarella	cheese
2	sausage	meat
2	pebbers	vegetable

with relation schema
R(Pizza, Topping, ToppingType)

and functional dependencies
Pizza, ToppingType $\rightarrow$ Topping

Topping $\rightarrow$ ToppingType

Normalize the relation schema and table to BCNF.

7.1.5 Normalization to BCNF gives tables with schemas R1(Pizza, Topping) and R2(Topping, ToppingType)

Pizza	Topping
1	mozarella
1	pepperoni
1	olives
2	mozarella
2	sausage
2	pebbers

Topping	ToppingType
mozarella	cheese
pepperoni	meat
olives	Vegetable
sausage	meat
pebbers	vegetable

Exercises

Please answer all exercises to demonstrate your Database Skills.

Pencil and Paper Exercises

Solutions are available at 11:45

Normalization

7.2.1 Violation of First Normal Form 1NF

Consider the relation schema:
Employees(EmpNo, Jobs)
where Jobs is a multivalued attribute containing one or more jobs.

Make changes needed to ensure 1NF.

7.2.2 Violation of Second Normal Form 2NF

Consider the relation schema:
Clients(ClientNo, SalesRepNo, CName, SName, Date)
where ClientNo and CName is the number and name of a client, SalesRepNo and SName is the number and name of a sales representative, and Date is the date where the sales representative was assigned to the client.

Explain why Clients is not in 2NF. Hint: First decide the minimal, non-trivial functional dependences.

Normalize Clients to 2NF.

7.2.3 Violation of Third Normal Form 3NF

Consider the relation schema:
Winners(Tournament, Year, Winner, Birthday)
where each row in the table tells who was the Winner of a particular Tournament in a particular Year. The Birthday of the Winner is also given.

Explain why Winners is not in 3NF and why that is a problem. Is Winners in 2NF? Normalize Winners to 3NF.

7.2.4 Violation of Fourth Normal Form 4NF

We are told that in a company, an employee can work on many projects and be employed in many departments. Departments as well as projects can have many employees.

Consider the relation schema:
Company(EmpNo, DepNo, ProjectNo)
with the dependency:
EmpNo →→ ProjectNo
Explain why Company is not in 4NF.

Normalize Company to 4NF.

7.2.5 Functional Dependencies

How many functional dependencies can be proposed for the schema: R(A, B, C, D, E)?

7.2.6 Candidate keys with Armstrong's rules

Use Armstrong's rules to propose a primary key for the following Relation Schema:
R(A, B, C, D, E, F, G, H)
with the following set of functional dependencies:
{A → BC, E → FG, AB → D, EG → H}
Please specify which of Armstrong's rules are used in each step.